

CasaTunes WebSocket Protocol Specification

Version: 1.7.0

Date: 2026-05-10

Status: Draft

Table of Contents

1. [Overview](#)
 2. [Connection Lifecycle](#)
 3. [Message Envelope](#)
 - [Request](#)
 - [Response](#)
 - [Error Response](#)
 - [Event Broadcast](#)
 4. [Error Codes](#)
 5. [Namespaces and Operations](#)
 - [core](#)
 - [server](#)
 - [avSwitch](#)
 - [mediaPlayer](#)
 6. [Subscriptions](#)
 7. [Data Types Reference](#)
 8. [Appendix A — Forms](#)
-

Overview

The CasaTunes WebSocket protocol enables clients to control and monitor a CasaTunes whole-house audio system. Communication is bidirectional: clients send **requests** and receive **responses**, and the server sends unsolicited **event broadcasts** to notify subscribed clients of state changes.

All messages are UTF-8 encoded JSON. The protocol is namespace-based, allowing logical grouping of operations (e.g., [avSwitch](#), [mediaPlayer](#)) and fine-grained event subscriptions.

Connection Lifecycle

Upon establishing a WebSocket connection, clients **must** follow this sequence:

1. [core.init](#) (*required*) — Identify the client to the server (name, version). Must be the first message sent after connecting.
2. [server.get](#) — Retrieve server configuration and status.
3. [avSwitch.zone.getAll](#) (system mode) or [avSwitch.stream.getAll](#) (streamer mode) — Retrieve all zones and inputs, or all streams. Check [server.get mode](#) to determine which call to make.

4. **mediaPlayer.getAll** — Retrieve now-playing state for all media players.
5. **core.subscribe** — Subscribe to event namespaces to keep state in sync.
6. **Issue operations as needed** — Rely on event broadcasts to update local state rather than polling.
7. **Respond to core.ping** — The server sends a ping every 60 seconds. The client must respond with **core.pong** within 10 seconds or the connection will be closed.

State Reload

Clients must perform a full state reload — repeating steps 1–5 above — in the following situations:

- **On reconnect** — after a disconnection and successful re-establishment of the WebSocket connection, the client must reload all state as the server configuration or zone state may have changed while the client was offline.
- **On server.changed event** — when a **server.changed** event is received, the client must reload all state by repeating steps 2–5 (re-sending **core.init** is not required as the session is already established). This ensures the client reflects any changes to server configuration, mode, or available zones and inputs.

Note: Clients should not assume that locally cached state remains valid after a reconnect or **server.changed** event. A full reload is the only safe way to guarantee consistency.

Init Enforcement Rules

- **core.init** is **mandatory** and must be the first operation sent after connecting.
- The server allows a fixed window of **5 seconds** after the WebSocket connection is established for the client to send **core.init**. If the timeout expires without receiving **init**, the server closes the connection without sending a response.
- If a client sends **any other operation before core.init**, the server returns an error response and then immediately closes the connection.

Error returned for premature operation:

```
{
  "id":    "<echoed from request, or null if unparseable>",
  "ns":    "core",
  "op":    "init",
  "error": {
    "code": 403,
    "message": "core.init must be the first operation after connecting"
  }
}
```

Message Envelope

Request

Sent by the client to invoke an operation.

```
{
  "id": "<string>",
  "ns": "<string>",
  "op": "<string>",
  "args": { }
}
```

Field	Type	Required	Description
id	string	Yes	Client-defined identifier used to correlate responses. Any string format is acceptable.
ns	string	Yes	Namespace (e.g., <code>core</code> , <code>server</code> , <code>avSwitch</code> , <code>mediaPlayer</code>).
op	string	Yes	Operation name within the namespace (e.g., <code>zone.setVolume</code>).
args	object	No	Operation arguments. Omitted entirely when the operation takes no arguments.

Example — with args:

```
{
  "id": "42",
  "ns": "avSwitch",
  "op": "zone.setVolume",
  "args": { "zoneId": "12", "volume": 50 }
}
```

Example — without args:

```
{
  "id": "43",
  "ns": "server",
  "op": "get"
}
```

Response

Sent by the server in reply to a client request.

```
{
  "id": "<string>",
  "ns": "<string>",
  "op": "<string>",
}
```

```
"data": { }
}
```

Field	Type	Description
id	string	Echoes the <code>id</code> from the originating request.
ns	string	Echoes the <code>ns</code> from the originating request.
op	string	Echoes the <code>op</code> from the originating request.
data	object	Response payload. Fields that are unavailable or not applicable are omitted rather than set to <code>null</code> .

Example:

```
{
  "id":    "42",
  "ns":    "avSwitch",
  "op":    "zone.setVolume",
  "data": { "zoneId": "12", "volume": 50 }
}
```

Error Response

Returned instead of a normal response when the operation fails.

```
{
  "id":    "<string>",
  "ns":    "<string>",
  "op":    "<string>",
  "error": {
    "code":    <number>,
    "message": "<string>"
  }
}
```

Field	Type	Description
id	string	Echoes the <code>id</code> from the originating request.
ns	string	Echoes the <code>ns</code> from the originating request.
op	string	Echoes the <code>op</code> from the originating request.
error.code	number	Numeric error code. See Error Codes .
error.message	string	Human-readable description of the error.

Example:

```
{
  "id":    "42",
  "ns":    "avSwitch",
  "op":    "zone.setVolume",
  "error": {
    "code": 404,
    "message": "ZoneID 12 does not exist"
  }
}
```

Event Broadcast

Sent by the server to all subscribed clients when state changes. Not associated with any client request.

```
{
  "type": "event",
  "ns":    "<string>",
  "event": "<string>",
  "data": { }
}
```

Field	Type	Description
type	string	Always "event" . Identifies this as an unsolicited broadcast.
ns	string	Namespace that originated the event.
event	string	Event name (e.g., zone.changed , server.changed).
data	object	Event payload. Always contains the complete current object.

Example:

```
{
  "type": "event",
  "ns":    "avSwitch",
  "event": "zone.changed",
  "data": { "zoneId": "12", "volume": 40 }
}
```

Parsing tip: Clients should check for **"type": "event"** first to distinguish event broadcasts from operation responses, since events do not carry an **id** or **op** field.

Error Codes

Code	Name	Description
400	Bad Request	The request message is malformed or missing required fields.
403	Forbidden	Operation rejected because <code>core.init</code> has not been completed.
404	Not Found	The specified resource (zone, input, etc.) does not exist.
405	Invalid Operation	The operation is not valid in the current server mode or state.
409	Conflict	The operation conflicts with the current state.
422	Invalid Argument	An argument value is out of range or of an unexpected type.
500	Internal Error	An unexpected server-side error occurred.
503	Service Unavailable	The server or a subsystem is not ready to handle requests.

Namespaces and Operations

`core` Namespace

Core operations manage the client session and subscription state.

`core.init`

Required. Must be the first operation sent after connecting. Identifies the client to the server and establishes the session. The server allows a 5-second window after connection for `core.init` to arrive; if the timeout expires, the connection is closed. Any other operation sent before `core.init` results in a `403` error response followed by connection closure.

Request args:

Field	Type	Required	Description
<code>clientName</code>	string	Yes	Human-readable name of the client application.
<code>clientVersion</code>	string	Yes	Version string of the client application.

Request:

```
{
  "id":    "1",
  "ns":    "core",
  "op":    "init",
  "args": { "clientName": "Crestron Driver", "clientVersion": "2.1.0" }
}
```

Response data:

Field	Type	Description
<code>serviceVersion</code>	string	Version of the CasaTunes service.

Response:

```
{
  "id":    "1",
  "ns":    "core",
  "op":    "init",
  "data": { "serviceVersion": "6.5.0" }
}
```

core.ping / core.pong

The server sends a `core.ping` to each connected client every **60 seconds**. The client must respond with a `core.pong` within **10 seconds**. If no pong is received within the grace period, the server closes the connection.

Server → Client ping:

```
{
  "type":  "event",
  "ns":    "core",
  "event": "ping"
}
```

Client → Server pong:

```
{
  "id":    "<any>",
  "ns":    "core",
  "op":    "pong"
}
```

Server response to pong: *(no response — pong is acknowledgement only)*

Note: The ping is sent as an event broadcast (no `id`), since it is server-initiated. The client's pong uses the standard request envelope. Clients that fail to respond within 10 seconds will be disconnected.

core.subscribe

Subscribes the client to one or more event topics. See [Subscriptions](#) for topic format.

Request args:

Field	Type	Required	Description
topics	array of string	Yes	List of topic strings to subscribe to.

Request:

```
{
  "id":    "3",
  "ns":    "core",
  "op":    "subscribe",
  "args": { "topics": ["avSwitch", "mediaPlayer"] }
}
```

Response data:

Field	Type	Description
subscribed	array of string	Topics that were successfully subscribed.
warnings	array of string	Topics that were not recognised and were ignored. Omitted if none.

Response:

```
{
  "id":    "3",
  "ns":    "core",
  "op":    "subscribe",
  "data": {
    "subscribed": ["avSwitch", "mediaPlayer"]
  }
}
```

Response (with unknown topic):

```
{
  "id":    "3",
  "ns":    "core",
  "op":    "subscribe",
  "data": {
    "subscribed": ["avSwitch"],
    "warnings":   ["unknownNamespace"]
  }
}
```

core.unsubscribe

Unsubscribes the client from one or more event topics.

Request args:

Field	Type	Required	Description
topics	array of string	Yes	List of topic strings to unsubscribe from.

Request:

```
{
  "id": "4",
  "ns": "core",
  "op": "unsubscribe",
  "args": { "topics": ["mediaPlayer"] }
}
```

Response data:

Field	Type	Description
unsubscribed	array of string	Topics successfully unsubscribed.
warnings	array of string	Topics that were not recognised or not previously subscribed. Omitted if none.

server Namespace

Provides server identity, configuration, and status.

server.get

Returns server information. Fields that are not applicable to the server's current mode are omitted from the response.

Request: (no args)

```
{ "id": "5", "ns": "server", "op": "get" }
```

Response data:

Field	Type	Description
name	string	Server name.

Field	Type	Description
<code>matrixModel</code>	string	Hardware model identifier.
<code>mac</code>	string	MAC address.
<code>ipAddress</code>	string	IP address.
<code>firmwareVersion</code>	string	Currently installed firmware version.
<code>latestFirmwareVersion</code>	string	Latest available firmware version. Omitted if no update information is available.
<code>mode</code>	string	Server mode: <code>"system"</code> or <code>"streamer"</code> .
<code>online</code>	boolean	<code>true</code> if the CasaTunes service is reachable and responding.
<code>ready</code>	boolean	<code>true</code> if the service is fully initialised and the WSS has a complete snapshot of all state (zones, inputs, now-playing). Clients should wait for <code>ready: true</code> before issuing control operations.
<code>startedOn</code>	string	ISO 8601 UTC timestamp of when the CasaTunes service started.
<code>sessionId</code>	string	Current session identifier. Changes when the server restarts or its configuration changes significantly. Clients should perform a full state reload when this value changes.
<code>numberOfStreams</code>	number	Number of licensed streams.
<code>numberOfZones</code>	number	Number of configured zones. Omitted in streamer mode.
<code>numberOfInputs</code>	number	Number of configured inputs. Omitted in streamer mode.
<code>problemList</code>	array	List of active problems. Each item is a <code>ProblemItem</code> object (see below). Empty array if none.

ProblemItem object:

Field	Type	Description
<code>problemId</code>	string	Unique identifier for this problem.
<code>code</code>	number	Numeric problem code.
<code>severity</code>	string	One of: <code>"info"</code> , <code>"warning"</code> , <code>"error"</code> .
<code>message</code>	string	Human-readable description of the problem.

Example:

```
"problemList": [  
  { "problemId": "FW001", "code": 100, "severity": "info",    "message":  
    "Firmware update available" },
```

```
{ "problemId": "NET001", "code": 200, "severity": "warning", "message":  
  "Internet connection unavailable" },  
  { "problemId": "ZN003", "code": 300, "severity": "error", "message":  
    "Zone 3 is unreachable" }  
]
```

server.message.respond

Sends the user's response to a `server.message` event. Only valid while the message is still active.

Request args:

Field	Type	Required	Description
<code>messageId</code>	string	Yes	The <code>messageId</code> from the <code>server.message</code> event.
<code>selection</code>	number	Yes	The <code>value</code> of the button selected by the user.

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the response was accepted.
<code>error</code>	string	Human-readable error message. Omitted on success.

server.tasks.get

Returns the list of available tasks.

Request: *(no args)*

Response data:

Field	Type	Description
<code>tasks</code>	array	Array of Task objects.

Task object:

Field	Type	Description
<code>taskId</code>	string	Unique task identifier.
<code>name</code>	string	Display name of the task.

server.task.invoke

Runs a task by name or ID.

Request args:

Field	Type	Required	Description
<code>task</code>	string	Yes	Task name or task ID.

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the task was successfully invoked.

`server.chimes.get`

Returns the list of available chime names. The returned names are used as the `chime` parameter in `server.chimes.play`.

Request: *(no args)*

Response data:

Field	Type	Description
<code>chimes</code>	array	Array of chime name strings.

Example:

```
{ "chimes": ["Chime 1", "Chime 2", "Westminster"] }
```

`server.chimes.play`

Plays a chime. The REST endpoint called depends on which of `zoneId` and `chime` are provided:

<code>zoneId</code>	<code>chime</code>	Behaviour
absent	absent	Plays the default chime in all rooms enabled for paging.
absent	present	Plays the specified chime in all rooms enabled for paging.
present	absent	Plays the default chime in the specified zone or zone group.
present	present	Plays the specified chime in the specified zone or zone group.

Request args:

Field	Type	Required	Description
<code>zoneId</code>	string	No	Target zone or zone group — numeric ID or name. Omit to play in all paging-enabled rooms.

Field	Type	Required	Description
<code>chime</code>	string	No	Chime name from <code>server.chimes.get</code> . Omit to play the default chime.
<code>preWait</code>	number	No	Seconds to wait before playing (allows matrix hardware to power on).
<code>postWait</code>	number	No	Seconds to wait after playing (prevents the chime from being cut off).
<code>volume</code>	number	No	Override playback volume in the target zone or zone group.

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the chime was successfully triggered.

`server.tts.play`

Plays text-to-speech audio. The REST endpoint called depends on whether `id` is provided:

<code>id</code>	Behaviour
absent	<code>GET /system/tts/input/{input}</code> — plays in all rooms enabled for paging; voice params are optional.
present	<code>POST /system/tts</code> — plays in the specified zone or zone group; voice params are included in the request body.

Request args:

Field	Type	Required	Description
<code>input</code>	string	Yes	The text to speak.
<code>id</code>	string	No	Target zone or zone group — numeric ID or name. Omit to play in all paging-enabled rooms.
<code>languageCode</code>	string	No	BCP-47 language code (e.g. <code>"en-US"</code>).
<code>gender</code>	string	No	Voice gender (e.g. <code>"MALE"</code> , <code>"FEMALE"</code> , <code>"NEUTRAL"</code>).
<code>voice</code>	string	No	Voice identifier (e.g. <code>"en-US-Wavenet-A"</code>).
<code>ssml</code>	boolean	No	<code>true</code> if <code>input</code> is SSML markup rather than plain text.
<code>preWait</code>	number	No	Seconds to wait before speaking.
<code>postWait</code>	number	No	Seconds to wait after speaking.
<code>volume</code>	number	No	Override playback volume.

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the TTS request was accepted.

server Events

server.changed — Fired when server configuration or session changes. Clients must perform a full state reload when received. See [State Reload](#).

```
{ "type": "event", "ns": "server", "event": "changed" }
```

server.issues.changed — Fired when the server problem list changes. `problemList` is `null` if all issues have been resolved.

```
{
  "type": "event",
  "ns": "server",
  "event": "issues.changed",
  "data": {
    "problemList": [
      { "problemId": "NET001", "code": 200, "severity": "warning",
"message": "Internet connection unavailable" }
    ]
  }
}
```

server.message — Fired when the CasaTunes service sends a notification message to be displayed to the user. The client should display the message and, if buttons are present, allow the user to respond via `server.message.respond`. A `null` message means any currently displayed message should be dismissed.

```
{
  "type": "event",
  "ns": "server",
  "event": "message",
  "data": {
    "messageId": "msg-001",
    "title": "Firmware Update Available",
    "body": "Version 6.6.0 is available. Would you like to update
now?",
    "duration": 30,
    "buttons": [
      { "value": 4, "title": "Yes" },

```

```
    { "value": 8, "title": "No" }
  ]
}
```

Message data fields:

Field	Type	Description
messageId	string	Identifier used when responding via <code>server.message.respond</code> .
title	string	Message title.
body	string	Message body text.
duration	number	Suggested display duration in seconds. <code>0</code> means display indefinitely until dismissed.
buttons	array	Array of button objects. Omitted if no response is required.

Button object:

Field	Type	Description
value	number	Value to pass as <code>selection</code> in <code>server.message.respond</code> : <code>1</code> = OK, <code>2</code> = Cancel, <code>4</code> = Yes, <code>8</code> = No.
title	string	Display label: <code>"OK"</code> , <code>"Cancel"</code> , <code>"Yes"</code> , or <code>"No"</code> .

avSwitch Namespace

Controls AV switch zones and inputs (system mode) or streams (streamer mode). Check `server.get mode` to determine which set of operations to use — **do not mix zone and stream calls**.

ID identification in requests: Any `zoneId` or `streamId` argument accepts either the numeric ID as a string (e.g. `"12"`) or the display name (e.g. `"Office"`, case-insensitive). IDs in responses and events are always the numeric ID string.

Zone Operations (System Mode)

Data Shapes

Input object:

Field	Type	Description
inputId	string	Unique identifier for the input.
name	string	Display name.

Field	Type	Description
type	string	Input type. One of: "casatunesMediaPlayer", "sonosPlayer", "unsupportedSource".
hidden	boolean	Whether the input is hidden from normal listings.

Zone object:

Field	Type	Description
zoneId	string	Numeric zone ID (e.g. "12"). Always numeric in responses and events.
name	string	Display name.
hidden	boolean	Whether the zone is hidden from normal listings.
power	boolean	true if powered on.
mute	boolean	true if muted.
volume	number	Current volume, 0–100.
inputId	string	Currently selected input.
maxVolume	number	Maximum allowed volume, 0–100. Always enforced.
turnOnVolume	object	{ "value": number, "enabled": boolean } — volume applied when the zone powers on.
fixedVolume	object	{ "value": number, "enabled": boolean } — fixed volume level. When enabled, volume controls are locked.
zonesInGroup	array	List of zoneId strings for each zone in the group. Empty if the zone is not grouped.
sleepEnabled	boolean	true if a sleep timer is currently active for this zone.

Example Zone object:

```
{
  "zoneId": "12",
  "name": "Office",
  "hidden": false,
  "power": true,
  "mute": false,
  "volume": 45,
  "inputId": "2",
  "maxVolume": 80,
  "turnOnVolume": { "value": 30, "enabled": true },
  "fixedVolume": { "value": 50, "enabled": false },
  "zonesInGroup": ["5", "9"],
  "sleepEnabled": false
}
```

avSwitch.zone.getAll

Returns all zones and inputs. Use in system mode only.

Request: *(no args)*

```
{ "id": "10", "ns": "avSwitch", "op": "zone.getAll" }
```

Response data:

Field	Type	Description
zones	array	Array of Zone objects.
inputs	array	Array of Input objects.

avSwitch.zone.get

Returns the full state of a single zone.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Zone to retrieve — numeric ID or name.

Response data: A single Zone object.

avSwitch.zone.setPower

Sets or toggles the power state of a zone.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Target zone — numeric ID or name.
power	string	Yes	"on", "off", or "toggle".

Response data:

Field	Type	Description
zoneId	string	Target zone.
power	boolean	Resulting power state.

avSwitch.zone.setMute

Sets or toggles the mute state of a zone.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Target zone — numeric ID or name.
mute	string	Yes	"on", "off", or "toggle".

Response data:

Field	Type	Description
zoneId	string	Target zone.
mute	boolean	Resulting mute state.

avSwitch.zone.setVolume

Sets the absolute volume of a zone.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Target zone — numeric ID or name.
volume	number	Yes	Target volume, 0–100. Clamped to zone's maxVolume if exceeded.

Response data:

Field	Type	Description
zoneId	string	Target zone.
volume	number	Resulting volume level.

avSwitch.zone.adjustVolume

Adjusts the volume of a zone by a signed delta. Positive values increase volume, negative values decrease it. The resulting volume is clamped to 0–100 and will not exceed the zone's maxVolume.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Target zone — numeric ID or name.

Field	Type	Required	Description
<code>delta</code>	number	Yes	Amount to adjust. Positive to increase, negative to decrease (e.g., <code>5</code> or <code>-5</code>).

Response data:

Field	Type	Description
<code>zoneId</code>	string	Target zone.
<code>volume</code>	number	Resulting volume level after adjustment.

Example:

```
{
  "id": "15",
  "ns": "avSwitch",
  "op": "zone.adjustVolume",
  "args": { "zoneId": "12", "delta": -5 }
}
```

`avSwitch.zone.setInput`

Sets the active input for a zone.

Request args:

Field	Type	Required	Description
<code>zoneId</code>	string	Yes	Target zone.
<code>inputId</code>	string	Yes	Input to select.

Response data:

Field	Type	Description
<code>zoneId</code>	string	Target zone.
<code>inputId</code>	string	Resulting active input.

`avSwitch.zone.setMaxVolume`

Sets the maximum volume ceiling for a zone. Always enforced — there is no enabled/disabled state.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Target zone.
value	number	Yes	Maximum volume, 0–100.

Request:

```
{
  "id":    "21",
  "ns":    "avSwitch",
  "op":    "zone.setMaxVolume",
  "args": { "zoneId": "12", "value": 80 }
}
```

Response data:

Field	Type	Description
zoneId	string	Target zone.
maxVolume	number	Resulting max volume.

avSwitch.zone.setTurnOnVolume

Sets the volume applied when the zone powers on.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Target zone.
value	number	Yes	Volume on power-on, 0–100.
enabled	boolean	Yes	true to apply the turn-on volume, false to disable it.

Request:

```
{
  "id":    "22",
  "ns":    "avSwitch",
  "op":    "zone.setTurnOnVolume",
  "args": { "zoneId": "12", "value": 30, "enabled": true }
}
```

Response data:

Field	Type	Description
zoneId	string	Target zone.
turnOnVolume	object	{ "value": number, "enabled": boolean } — resulting state.

avSwitch.zone.setFixedVolume

Sets a fixed volume level for a zone. When enabled, volume controls are locked at this level.

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Target zone.
value	number	Yes	Fixed volume level, 0–100.
enabled	boolean	Yes	true to lock volume at this level, false to disable.

Request:

```
{
  "id": "24",
  "ns": "avSwitch",
  "op": "zone.setFixedVolume",
  "args": { "zoneId": "12", "value": 50, "enabled": true }
}
```

Response data:

Field	Type	Description
zoneId	string	Target zone.
fixedVolume	object	{ "value": number, "enabled": boolean } — resulting state.

avSwitch.zone.group

Links a zone to another zone, adding it to that zone's group. The zone is powered on and assigned the same source as the target zone. Returns **success: false** if CasaTunes cannot complete the join (e.g. the target zone is not powered on or does not exist).

Request args:

Field	Type	Required	Description
zoneId	string	Yes	Zone to add to the group — numeric ID or name.
targetZoneId	string	Yes	Zone whose group to join — numeric ID or name.

Request:

```
{
  "id":    "25",
  "ns":    "avSwitch",
  "op":    "zone.group",
  "args": { "zoneId": "9", "targetZoneId": "5" }
}
```

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the zone was successfully linked.

avSwitch.zone.ungroup

Removes a zone from its group by powering it off.

Request args:

Field	Type	Required	Description
<code>zoneId</code>	string	Yes	Zone to remove from its group — numeric ID or name.

Request:

```
{
  "id":    "26",
  "ns":    "avSwitch",
  "op":    "zone.ungroup",
  "args": { "zoneId": "9" }
}
```

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the zone was successfully powered off.

avSwitch.zone.setSleepTimer

Powers off a zone with an optional delay. A `delay` of `0` powers the zone off immediately. A positive `delay` schedules the power-off after the specified number of seconds. The `sleepEnabled` field in the response reflects whether a sleep timer is now active.

Request args:

Field	Type	Required	Description
<code>zoneId</code>	string	Yes	Target zone — numeric ID or name.
<code>delay</code>	number	Yes	Sleep delay in seconds. <code>0</code> = power off immediately. Must be ≥ 0 .

Response data:

Field	Type	Description
<code>zoneId</code>	string	Target zone.
<code>power</code>	boolean	Resulting power state.
<code>sleepEnabled</code>	boolean	<code>true</code> if a sleep timer is now active.

Stream Operations (Streamer Mode)

In streamer mode each stream corresponds to a single audio output. Its `streamId` equals its `sourceId`. Streams are always powered on; volume may be fixed or variable. Use `avSwitch.stream.*` methods in streamer mode only.

Stream object:

Field	Type	Description
<code>streamId</code>	string	Numeric stream ID. Equals the corresponding <code>sourceId</code> .
<code>name</code>	string	Display name.
<code>hidden</code>	boolean	Whether the stream is hidden from normal listings.
<code>mute</code>	boolean	<code>true</code> if muted.
<code>volume</code>	number	Current volume, 0–100.
<code>maxVolume</code>	number	Maximum allowed volume, 0–100.
<code>fixedVolume</code>	object	<code>{ "value": number, "enabled": boolean }</code> — when enabled, volume controls are locked at this level.

`avSwitch.stream.getAll`

Returns all streams. Use in streamer mode only.

Request: *(no args)*

```
{ "id": "10", "ns": "avSwitch", "op": "stream.getAll" }
```

Response data:

Field	Type	Description
<code>streams</code>	array	Array of Stream objects.

`avSwitch.stream.get`

Returns the full state of a single stream.

Request args:

Field	Type	Required	Description
<code>streamId</code>	string	Yes	Stream to retrieve — numeric ID or name.

Response data: A single Stream object.

`avSwitch.stream.setMute`

Sets or toggles the mute state of a stream.

Request args:

Field	Type	Required	Description
<code>streamId</code>	string	Yes	Target stream — numeric ID or name.
<code>mute</code>	string	Yes	"on", "off", or "toggle".

Response data:

Field	Type	Description
<code>streamId</code>	string	Target stream.
<code>mute</code>	boolean	Resulting mute state.

`avSwitch.stream.setVolume`

Sets the absolute volume of a stream.

Request args:

Field	Type	Required	Description
<code>streamId</code>	string	Yes	Target stream — numeric ID or name.
<code>volume</code>	number	Yes	Target volume, 0–100. Clamped to stream's <code>maxVolume</code> if exceeded.

Response data:

Field	Type	Description
<code>streamId</code>	string	Target stream.
<code>volume</code>	number	Resulting volume level.

`avSwitch.stream.adjustVolume`

Adjusts the volume of a stream by a signed delta. The resulting volume is clamped to 0–100 and will not exceed the stream's `maxVolume`.

Request args:

Field	Type	Required	Description
<code>streamId</code>	string	Yes	Target stream — numeric ID or name.
<code>delta</code>	number	Yes	Amount to adjust. Positive to increase, negative to decrease (e.g., <code>5</code> or <code>-5</code>).

Response data:

Field	Type	Description
<code>streamId</code>	string	Target stream.
<code>volume</code>	number	Resulting volume level after adjustment.

`avSwitch.stream.setMaxVolume`

Sets the maximum volume ceiling for a stream.

Request args:

Field	Type	Required	Description
<code>streamId</code>	string	Yes	Target stream — numeric ID or name.
<code>value</code>	number	Yes	Maximum volume, 0–100.

Response data:

Field	Type	Description
<code>streamId</code>	string	Target stream.
<code>maxVolume</code>	number	Resulting max volume.

`avSwitch.stream.setFixedVolume`

Sets a fixed volume level for a stream. When enabled, volume controls are locked at this level.

Request args:

Field	Type	Required	Description
streamId	string	Yes	Target stream — numeric ID or name.
value	number	Yes	Fixed volume level, 0–100.
enabled	boolean	Yes	true to lock volume at this level, false to allow variable volume.

Response data:

Field	Type	Description
streamId	string	Target stream.
fixedVolume	object	{ "value": number, "enabled": boolean } — resulting state.

avSwitch Events

avSwitch.changed — Fired when any property of a zone or stream changes. **data** contains the complete Zone object (system mode) or Stream object (streamer mode).

```
{
  "type": "event",
  "ns": "avSwitch",
  "event": "changed",
  "data": {
    "zoneId": "12", "name": "Office", "power": true, "volume": 40,
    "mute": false, "inputId": "2", "maxVolume": 80,
    "turnOnVolume": { "value": 30, "enabled": true },
    "fixedVolume": { "value": 0, "enabled": false },
    "zonesInGroup": ["5", "9"],
    "hidden": false, "sleepEnabled": false
  }
}
```

avSwitch.group.changed — Fired when group membership changes for one or more zones (zones are linked/unlinked or powered on/off). **data** contains an array of the affected groups in their updated state. A group whose last member has left will appear with an empty **zones** array.

```
{
  "type": "event",
  "ns": "avSwitch",
  "event": "group.changed",
  "data": {
    "groups": [
      { "sourceId": "2", "zoneIds": ["5", "9"] }
    ]
  }
}
```

```
    ]
  }
}
```

Group changed data fields:

Field	Type	Description
groups	array	Array of group objects for each group whose membership changed.

Group object:

Field	Type	Description
sourceId	string	The source shared by all zones in the group.
zoneIds	array	Array of zoneId strings. Empty if the group dissolved.

avSwitch.input.changed — Fired when any property of an input changes. data contains the complete Input object.

```
{
  "type": "event",
  "ns": "avSwitch",
  "event": "input.changed",
  "data": { "inputId": "2", "name": "Spotify", "type":
"casatunesMediaPlayer", "hidden": false }
}
```

mediaPlayer Namespace

Controls media playback for audio sources (inputs).

Data Shapes

NowPlaying object:

Field	Type	Description
inputId	string	Input this player is associated with.
mediaId	string	Identifier of the currently playing media item.
artworkUrl	string	URL of the current track or station artwork. Omitted if unavailable.
metaData	array	Ordered list of display strings describing the current track. Render in the order provided. See note below.
serviceInfo	object	Music service information. See below.

Field	Type	Description
<code>searchPromptText</code>	string	Hint text for a search input scoped to the current source. Omitted if search is not available.
<code>duration</code>	number	Total track duration in seconds. <code>-1</code> if unavailable.
<code>progress</code>	number	Current playback position in seconds. <code>-1</code> if unavailable.
<code>status</code>	number	Playback status: <code>0</code> = stopped, <code>1</code> = paused, <code>2</code> = playing.
<code>featured</code>	object	<code>{ "isAvailable": boolean, "value": string }</code> — value is "on" or "off".
<code>jump</code>	object	<code>{ "isAvailable": boolean }</code> — whether the user can jump forward/backward within the current track (e.g. podcast skip).
<code>next</code>	object	<code>{ "isAvailable": boolean, "isEnabled": boolean }</code> — <code>isAvailable</code> indicates the source supports skip-next; <code>isEnabled</code> indicates there is currently a next track to skip to.
<code>pause</code>	object	<code>{ "isAvailable": boolean }</code> — whether the pause action can be performed.
<code>play</code>	object	<code>{ "isAvailable": boolean }</code> — whether the play action can be performed.
<code>previous</code>	object	<code>{ "isAvailable": boolean, "isEnabled": boolean }</code> — <code>isAvailable</code> indicates the source supports skip-previous; <code>isEnabled</code> indicates skip-previous is currently actionable.
<code>progressBar</code>	object	<code>{ "isAvailable": boolean }</code> — whether a progress bar should be displayed for this source.
<code>queue</code>	object	<code>{ "isAvailable": boolean }</code> — whether the queue is supported for this source.
<code>repeat</code>	object	<code>{ "isAvailable": boolean, "value": string }</code> — value is "on", "once", or "off".
<code>search</code>	object	<code>{ "isAvailable": boolean }</code> — whether search is available for this source.
<code>seek</code>	object	<code>{ "isAvailable": boolean }</code> — whether the user can seek to an absolute position within the current track.
<code>shuffle</code>	object	<code>{ "isAvailable": boolean, "value": string }</code> — value is "on" or "off".
<code>stop</code>	object	<code>{ "isAvailable": boolean }</code> — whether the stop action can be performed.
<code>thumbsDown</code>	object	<code>{ "isAvailable": boolean, "value": boolean }</code> — <code>isAvailable</code> indicates the source supports thumbs rating; <code>value</code> is <code>true</code> if the current track is rated down.

Field	Type	Description
<code>thumbsUp</code>	object	<code>{ "isAvailable": boolean, "value": boolean }</code> — <code>isAvailable</code> indicates the source supports thumbs rating; <code>value</code> is <code>true</code> if the current track is rated up.
<code>hasQueueItems</code>	boolean	<code>true</code> if the queue contains one or more items.

Example NowPlaying object:

```
{
  "inputId":      "1",
  "mediaId":      "12326726412843",
  "artworkUrl":   "https://example.com/artwork.jpg",
  "metaData":     ["Blue Sky", "The Allman Brothers Band", "Eat a Peach"],
  "serviceInfo": {
    "serviceName": "Pandora",
    "serviceLogoUrl": "https://example.com/pandora-logo.png",
    "accountName":  "david@example.com",
  },
  "duration": 318,
  "progress": 42,
  "status": 2,
  "featured": { "isAvailable": true, "value": "on" },
  "jump": { "isAvailable": false },
  "next": { "isAvailable": true, "isEnabled": true },
  "pause": { "isAvailable": true },
  "play": { "isAvailable": false },
  "previous": { "isAvailable": true, "isEnabled": false },
  "progressBar": { "isAvailable": true },
  "queue": { "isAvailable": true },
  "repeat": { "isAvailable": true, "value": "on" },
  "search": { "isAvailable": true },
  "seek": { "isAvailable": true },
  "shuffle": { "isAvailable": true, "value": "off" },
  "stop": { "isAvailable": true },
  "thumbsDown": { "isAvailable": true, "value": false },
  "thumbsUp": { "isAvailable": true, "value": false },
  "hasQueueItems": false
}
```

metaData note: `metaData` is an ordered array of strings. Clients should display the items in the order provided without interpreting individual fields. The server will populate as many of the following as are available for the current service: song title, artist, album, playlist name. Items that are unavailable are omitted rather than included as empty strings.

Example for a track with full metadata:

```
"metaData": ["Blue Sky", "The Allman Brothers Band", "Eat a Peach"]
```

Example for a streaming radio station with only a station description:

```
"metaData": ["Classic Rock Radio"]
```

ServiceInfo object:

Field	Type	Description
<code>serviceName</code>	string	Name of the music service (e.g., "Spotify").
<code>serviceLogoUrl</code>	string	URL of the service logo.
<code>accountName</code>	string	Name of the logged-in account.

`mediaPlayer.getAll`

Returns now-playing information for all media player inputs.

Request: *(no args)*

```
{ "id": "20", "ns": "mediaPlayer", "op": "getAll" }
```

Response data:

Field	Type	Description
<code>players</code>	array	Array of NowPlaying objects, one per input.

`mediaPlayer.get`

Returns now-playing information for a specific input.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Input to retrieve.

Response data: A single NowPlaying object.

`mediaPlayer.play`

Starts playback on an input.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.pause`

Pauses playback on an input.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.playPause`

Toggles between play and pause. If currently playing, pauses; if paused or stopped, plays.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.stop`

Stops playback on an input.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.next`

Skips to the next track.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.previous`

Returns to the previous track.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.shuffle`

Sets shuffle mode.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>mode</code>	string	Yes	"on" or "off".

Response data: A single NowPlaying object.

`mediaPlayer.repeat`

Sets the repeat mode.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>mode</code>	string	Yes	"on" (repeat all), "once" (repeat queue one more time), or "off".

Response data: A single NowPlaying object.

`mediaPlayer.featured`

Toggles or sets whether the current input is playing from featured content.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>mode</code>	string	Yes	"on", "off", or "toggle".

Response data: A single NowPlaying object.

`mediaPlayer.queue.clear`

Clears the playback queue.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.queue.get`

Returns the contents of the playback queue for an input. Supports pagination for large queues.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>limit</code>	number	No	Maximum number of items to return.
<code>offset</code>	number	No	Zero-based start index for pagination.

Response data: A single Queue object.

`mediaPlayer.queue.save`

Saves the current playback queue as a named playlist.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>name</code>	string	Yes	Name for the new playlist.

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the playlist was saved successfully.
<code>error</code>	string	Human-readable error message. Omitted on success.

`mediaPlayer.queue.playItem`

Starts playback from a specific position in the queue. The queue contents are unchanged — only the current play position advances. State updates are delivered via the `mediaPlayer.changed` event.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>index</code>	number	Yes	Zero-based index of the queue item to play.

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the operation succeeded.
<code>error</code>	string	Human-readable error message. Omitted on success.

`mediaPlayer.queue.deleteItem`

Removes an item from the playback queue by index. The client may optimistically remove the item locally rather than re-fetching the full queue.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>index</code>	number	Yes	Zero-based index of the queue item to remove.

Response data:

Field	Type	Description
<code>success</code>	boolean	<code>true</code> if the item was removed successfully.
<code>error</code>	string	Human-readable error message. Omitted on success.

`mediaPlayer.thumbsUp`

Sends a thumbs-up rating for the currently playing track.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.thumbsDown`

Sends a thumbs-down rating for the currently playing track.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.

Response data: A single NowPlaying object.

`mediaPlayer.position`

Seeks to an absolute position within the current track.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>position</code>	number	Yes	Absolute position in seconds from the start of the track.

Response data: A single NowPlaying object.

`mediaPlayer.jump`

Adjusts the playback position by a relative offset. Positive values jump forward, negative values jump backward.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>delta</code>	number	Yes	Relative offset in seconds. Positive to jump forward, negative to jump backward.

Response data: A single NowPlaying object.

mediaPlayer.featured.get

Returns featured or curated media items for an input.

Request args:

Field	Type	Required	Description
inputId	string	Yes	Target input.
start	number	No	Zero-based offset for pagination. Defaults to 0.
maxItems	number	No	Maximum number of items to return.

Request:

```
{
  "id": "30",
  "ns": "mediaPlayer",
  "op": "featured.get",
  "args": { "inputId": "1", "start": 0, "maxItems": 20 }
}
```

Response data:

Field	Type	Description
featuredItems	array	Array of FeaturedItem objects.

FeaturedItem object:

Field	Type	Description
mediaId	string	Media identifier, usable with mediaPlayer.media.play.
title	string	Display title (e.g., "Today's Hits").
type	string	Content type description (e.g., "Pandora Station", "Spotify Playlist").
artworkUrl	string	URL of the artwork to display for this item. Omitted if unavailable.

mediaPlayer.media.getRoot

Returns the top-level browsable collection for an input. This is the entry point for all media browsing — call this first to get the root menu of services, playlists, bookmarks, and history available for the input.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>includePlaylists</code>	boolean	No	Whether to include playlists.
<code>maxPlaylists</code>	number	No	Maximum number of playlists to return.
<code>includeOtherPlaylists</code>	boolean	No	Whether to include playlists from other sources.
<code>maxBookmarks</code>	number	No	Maximum number of bookmarks to return.
<code>includeSelectionHistory</code>	boolean	No	Whether to include recently played items.

Response data: A single Collection object.

`mediaPlayer.media.getCollection`

Browses into a collection item. Call this with a `mediaId` returned from `media.getRoot` or a previous `media.getCollection` to navigate deeper into the media hierarchy. Supports pagination via `limit` and `offset`.

Request args:

Field	Type	Required	Description
<code>mediaId</code>	string	Yes	Collection to browse into (from a previous browse response).
<code>inputId</code>	string	No	Input currently in use. When provided, the server uses the input's now-playing state to compute <code>canAddToQueue</code> .
<code>limit</code>	number	No	Maximum number of items to return.
<code>offset</code>	number	No	Zero-based start index for pagination.
<code>enableFormProcessing</code>	boolean	No	Default <code>false</code> . When <code>false</code> , if the server returns a form (e.g. for authentication), the form is suppressed and a single informational <code>MediaItem</code> is appended to the items list instead. Its title is " <code>{collection.title}</code> using CasaTunesX", or "Check the CasaTunesX App for more information" if the collection has no title. All capability flags on this item are <code>false</code> — it cannot be played, selected, or added to the queue. Set to <code>true</code> only for UIs that implement full form handling.

Response data: A single Collection object.

mediaPlayer.media.search

Searches within a collection. The `mediaId` scopes the search (e.g. search within a specific service or playlist). Returns a Collection object containing matching items.

Request args:

Field	Type	Required	Description
<code>mediaId</code>	string	Yes	Collection to search within.
<code>searchText</code>	string	Yes	Text to search for.
<code>inputId</code>	string	No	Input currently in use. When provided, the server uses the input's now-playing state to compute <code>canAddToQueue</code> .
<code>limit</code>	number	No	Maximum number of results to return.
<code>offset</code>	number	No	Zero-based start index for pagination.

Response data: A single Collection object.

mediaPlayer.media.submitForm

Submits a form in response to the user pressing a button. The client always POSTs using the pressed button's `id`. The body contains all form fields with user-supplied values. See [Appendix A — Forms](#) for the full interaction model.

Request args:

Field	Type	Required	Description
<code>buttonId</code>	string	Yes	The <code>id</code> of the button that was pressed.
<code>fields</code>	array	Yes	Array of FormField objects — every field in the form, with <code>value</code> set to the user-entered value. Hidden fields must be included with their original server-provided value unchanged.
<code>inputId</code>	string	No	Input currently in use. When provided, the server uses the input's now-playing state to compute <code>canAddToQueue</code> on the returned Collection.

Response data: A Collection object on success. On cancellation the response contains `error.symbol == "FormCancel"` — the client should close the form and return to the previous browsing state. On a validation error the response contains an `error` with a user-displayable `message` — the client should display the message and keep the form open.

mediaPlayer.media.play

Plays a specific media item on an input.

Request args:

Field	Type	Required	Description
<code>inputId</code>	string	Yes	Target input.
<code>mediaId</code>	string	Yes	Media item to play (from featured, search, or queue).
<code>addToQueue</code>	string	Yes	How the item is added to the queue. See values below.

`addToQueue` values:

Value	Description
<code>playNow</code>	Replace queue with this item and play immediately.
<code>playShuffle</code>	Same as <code>playNow</code> but with shuffle enabled.
<code>playUnshuffle</code>	Same as <code>playNow</code> but with shuffle disabled.
<code>add</code>	Add item to the queue without changing playback.
<code>addplay</code>	Add item to the queue and start playing the first newly added item.

Request:

```
{
  "id": "31",
  "ns": "mediaPlayer",
  "op": "media.play",
  "args": { "inputId": "1", "mediaId": "12326726412843", "addToQueue":
    "playNow" }
}
```

Response data: A single `NowPlaying` object.

`mediaPlayer.media.refresh`

Triggers an asynchronous server-side refresh of a collection. The response is returned immediately; the actual refresh happens in the background. Only applicable when `canRefresh` is `true` on the collection or item.

Request args:

Field	Type	Required	Description
<code>mediaId</code>	string	Yes	Collection or item to refresh.

Response data: `{ "success": true }`

mediaPlayer.media.delete

Deletes a media item or collection. Only applicable when `canDelete` is `true`. After a successful delete the client should re-fetch the parent collection.

Request args:

Field	Type	Required	Description
<code>mediaId</code>	string	Yes	Item or collection to delete.

Response data: { `"success": true` }

mediaPlayer.media.rename

Renames a media item or collection. Only applicable when `canRename` is `true`. After a successful rename the client should re-fetch the current collection to reflect the updated name.

Request args:

Field	Type	Required	Description
<code>mediaId</code>	string	Yes	Item or collection to rename.
<code>name</code>	string	Yes	New display name.

Response data: { `"success": true` }

mediaPlayer.media.setFeatured

Bookmarks or un-bookmarks a media item or collection. Only applicable when `canFeature` is `true`.

Request args:

Field	Type	Required	Description
<code>mediaId</code>	string	Yes	Item or collection to feature or un-feature.
<code>featured</code>	boolean	Yes	<code>true</code> to bookmark/feature; <code>false</code> to remove.

Response data: { `"success": true` }

mediaPlayer Events

mediaPlayer.changed — Fired when the now-playing state of an input changes. `data` contains the complete NowPlaying object.

Progress bar guidance: The `progress` field in this event is the authoritative re-sync point. Clients should not wait for server-sent progress updates — instead, maintain a local 1-second timer that increments `progress` automatically. The timer should run only when `status == 2`

(playing) and `progressBar.isAvailable == true`. The displayed value must be clamped to `duration` and the timer stopped once `duration` is reached. For live or streaming sources, `duration` is `-1` and `progressBar.isAvailable` is `false` — do not run the timer. The `mediaPlayer.changed` event re-syncs progress whenever a track changes, playback starts or stops, or a seek/jump operation is performed.

```
{
  "type": "event",
  "ns": "mediaPlayer",
  "event": "changed",
  "data": {
    "inputId": "1",
    "mediaId": "12326726412843",
    "metaData": ["Blue Sky", "The Allman Brothers Band", "Eat a Peach"],
    "status": 2,
    "duration": 318,
    "progress": 42,
    "featured": { "isAvailable": true, "value": "on" },
    "jump": { "isAvailable": false },
    "next": { "isAvailable": true, "isEnabled": true },
    "pause": { "isAvailable": true },
    "play": { "isAvailable": false },
    "previous": { "isAvailable": true, "isEnabled": false },
    "progressBar": { "isAvailable": true },
    "queue": { "isAvailable": true },
    "repeat": { "isAvailable": true, "value": "on" },
    "search": { "isAvailable": true },
    "seek": { "isAvailable": true },
    "shuffle": { "isAvailable": true, "value": "off" },
    "stop": { "isAvailable": true },
    "thumbsDown": { "isAvailable": true, "value": false },
    "thumbsUp": { "isAvailable": true, "value": false },
    "hasQueueItems": false
  }
}
```

mediaPlayer.featured.changed — Fired when the featured items for any input change. Clients should re-fetch via `mediaPlayer.featured.get` for the relevant input.

```
{
  "type": "event",
  "ns": "mediaPlayer",
  "event": "featured.changed"
}
```

Subscriptions

Clients subscribe to topics using `core.subscribe`. A topic identifies a namespace or a specific resource within a namespace.

Topic formats:

Topic	Description
<code>avSwitch</code>	All avSwitch zone events.
<code>avSwitch.zoneId</code>	Events for a specific zone only (future).
<code>mediaPlayer</code>	All mediaPlayer events.
<code>mediaPlayer.inputId</code>	Events for a specific input only (future).
<code>server</code>	All server events.

Note: Fine-grained topic filtering (e.g., `avSwitch.zoneId`) is reserved for future implementation. Currently, namespace-level subscriptions cover all events within that namespace.

Subscriptions persist for the lifetime of the WebSocket connection and are cleared automatically on disconnect.

Data Types Reference

Type	Description
string	JSON string.
number	JSON number (integer or float as indicated).
boolean	JSON <code>true</code> or <code>false</code> .
array	JSON array <code>[]</code> .
object	JSON object <code>{}</code> .

Volume range: 0–100 (integer). Fractional values are not used.

Timestamps: ISO 8601 strings (e.g., `"2026-02-24T10:00:00Z"`).

id format: Any client-defined string. Recommended to be unique per session to simplify response correlation.

Queue object

Returned by `mediaPlayer.queue.get`.

Field	Type	Description
<code>startIndex</code>	number	Zero-based index of the first item in <code>items</code> relative to the full queue.
<code>totalAvailable</code>	number	Total number of items in the queue (for pagination).
<code>items</code>	array	Array of <code>MediaItem</code> objects.

Collection object

Returned by `media.getRoot`, `media.getCollection`, `media.search`, and `media.submitForm`.

Field	Type	Description
<code>collectionId</code>	string	Identifier for this collection, usable with <code>media.getCollection</code> .
<code>title</code>	string	Display title for this collection.
<code>description</code>	string	Description of the collection. Omitted if unavailable.
<code>artworkUrl</code>	string	Artwork URL for this collection. Omitted if unavailable.
<code>canPlay</code>	boolean	Whether the entire collection can be played.
<code>canAddToQueue</code>	boolean	Whether the entire collection can be added to the queue. Requires <code>inputId</code> to be provided in the request; <code>false</code> if omitted.
<code>canSearch</code>	boolean	Whether this collection supports search.
<code>canRefresh</code>	boolean	Whether this collection can be refreshed.
<code>canDelete</code>	boolean	Whether this collection (or its items) can be deleted.
<code>canRename</code>	boolean	Whether this collection can be renamed.
<code>isFeatured</code>	boolean	<code>true</code> if this collection has been bookmarked/featured by the user.
<code>canFeature</code>	boolean	<code>true</code> if the user can bookmark/feature or un-feature this collection.
<code>searchPromptText</code>	string	Hint text for a search input. Only present when <code>canSearch</code> is <code>true</code> .
<code>startIndex</code>	number	Zero-based index of the first item in <code>items</code> relative to the full collection.
<code>totalAvailable</code>	number	Total number of items available in the full collection (for pagination).
<code>items</code>	array	Array of <code>Medialtem</code> objects.
<code>form</code>	object	Form object to display for authentication or data entry. Omitted if not required. See Form object below.
<code>error</code>	object	Error detail if the collection could not be loaded. <code>{ "code": number, "message": string }</code> . Omitted on success.

Medialtem object

Represents a single item within a Collection.

Field	Type	Description
<code>mediaId</code>	string	Identifier for this item, usable with <code>media.getCollection</code> (if browsable) or <code>media.play</code> (if playable).
<code>title</code>	string	Primary display title.
<code>details</code>	string	Secondary display text (e.g. artist, subtitle). Omitted if unavailable.
<code>artworkUrl</code>	string	Artwork URL. Omitted if unavailable.
<code>type</code>	number	Item type code. Determines whether the item is playable, browsable, or both.
<code>flags</code>	number	Capability flags bitmask.
<code>duration</code>	number	Track duration in seconds. <code>0</code> if not applicable.
<code>displayInfo</code>	array	Ordered array of display strings. Render in the order provided.
<code>isCollection</code>	boolean	<code>true</code> if this item is a browsable collection (can be opened with <code>mediaPlayer.media.getCollection</code>).
<code>isBrowseMore</code>	boolean	<code>true</code> if this item is a pagination sentinel — display as a "Browse more..." affordance rather than a regular item. To load the next page, call <code>mediaPlayer.media.getCollection</code> with the <code>mediaId</code> of this item and append the returned <code>items</code> to the existing collection item list.
<code>isDisabled</code>	boolean	<code>true</code> if this item is unplayable or unavailable (deleted, permission denied, etc.). Display as greyed out.
<code>isFeatured</code>	boolean	<code>true</code> if this item has been bookmarked/featured by the user.
<code>canFeature</code>	boolean	<code>true</code> if the user can bookmark/feature or un-feature this item.
<code>canDelete</code>	boolean	<code>true</code> if this item can be deleted.
<code>canRefresh</code>	boolean	<code>true</code> if this item supports a refresh action.
<code>canDisplayInfo</code>	boolean	<code>true</code> if this item has secondary display text available (<code>details</code> is non-empty).

Form object

Returned inside a Collection when a service requires user input (e.g. authentication or search). Submission is driven by button IDs — there is no separate form-level ID.

Field	Type	Description
<code>fields</code>	array	Array of FormField objects. May be empty for notice-style forms.
<code>buttons</code>	array	Array of FormButton objects.

FormField object:

Field	Type	Description
<code>key</code>	string	Field identifier, used as the key in <code>media.submitForm</code> pairs.
<code>title</code>	string	Display label.
<code>description</code>	string	Additional help text. Omitted if unavailable.
<code>type</code>	string	Input type (e.g. <code>"text"</code> , <code>"password"</code> , <code>"select"</code>).
<code>required</code>	boolean	Whether the field must be filled before submitting.
<code>value</code>	string	Pre-populated default value. Omitted if none.
<code>options</code>	array	Array of <code>{ "key": string, "value": string }</code> for <code>"select"</code> type fields. Omitted otherwise.

FormButton object:

Field	Type	Description
<code>key</code>	string	Button identifier (e.g. <code>"submit"</code> , <code>"cancel"</code> , <code>"signup"</code>).
<code>title</code>	string	Display label.
<code>id</code>	string	Endpoint identifier. Always present. The client POSTs to <code>media.submitForm</code> using this value as <code>buttonId</code> when the button is pressed.
<code>url</code>	string	External URL. Omitted if not applicable. When present, the client opens this URL in a new browser tab in addition to POSTing.

Appendix A — Forms

Forms allow music services to request user input. They are used for authentication (logging in to a streaming service) and for music service-specific operations such as adding a track to a playlist, or adding and removing tracks, albums, and playlists from a service's favorites. They are delivered as part of a Collection and require specific rendering and submission behaviour that differs from normal media browsing.

When a Form is Present

When `collection.form` is non-null, the client must render the form UI instead of the media item list. The `items` array will be empty.

Field Types

Type	Rendering	Notes
<code>text</code>	Standard text input	

Type	Rendering	Notes
password	Text input with obscured characters	Must be cleared after every submission attempt
select	Single-choice dropdown	Build options from <code>field.options</code> — use <code>key</code> as the submitted value, <code>value</code> as the display label
hidden	Not rendered	Must be included in every submission with its original server-provided value unchanged

Fields with a `value` property set by the server should be pre-populated in the UI.

Validation

- Any button whose `key` is not `"cancel"` or `"signup"` must be **disabled** until all fields with `required: true` are non-empty.
- Button enabled/disabled state must update **reactively** as the user types — do not wait for an explicit validate action.
- `hidden` fields do not participate in validation; they are always submitted.
- Error messages from a previous submission attempt must be cleared when the user modifies any field.

Submitting a Form

When the user presses any button, the client must:

- Always POST** via `media.submitForm` using the button's `id` as `buttonId`, regardless of which button was pressed (including cancel).
- Send **all fields** in the form as the `fields` body — every field, in order, with `value` set to the user-entered value. Hidden fields are included with their original value.
- If the button also has a `url`, **open that URL in a new browser tab** in addition to posting. This is used for OAuth and external signup flows.

Handling the Response

Condition	Client action
Response <code>error.symbol == "FormCancel"</code>	Close the form; return to the previous browsing state
Response has <code>error</code> (other symbol)	Display <code>error.message</code> inline; keep the form open for correction
Success (no error), button has <code>url</code>	The URL was already opened; close the form dialog
Success (no error), no <code>url</code> , returned Collection has <code>form</code>	Replace the current form with the new form and continue the interaction

Condition	Client action
Success (no error), no <code>url</code> , returned <code>collectionId</code> matches current view	Close the form; refresh the current collection view with the returned Collection
Success (no error), no <code>url</code> , returned <code>collectionId</code> differs from current view	Close the form; navigate to the returned Collection

Example: OAuth (Amazon Music Login)

Some services use OAuth rather than credential entry. The form has no input fields — only buttons. One button has both an `id` and a `url`.

Button press:

1. Client POSTs via `media.submitForm` using the button's `id`.
2. Client opens the button's `url` in a new browser tab, directing the user to the service's OAuth login page.
3. After the user authenticates externally, the service redirects back and the login completes. The client may receive a state-change event indicating the source is now authenticated.